软件工程·实验二

软件设计实验报告

城市外卖点餐与配送平台

——架构建模与界面设计

项目名称：　城市外卖点餐与配送平台
学　　号：　241220075
姓　　名：　赵力丹
设计输入：　实验一《需求分析》（REQ-001 至 REQ-026）
交付内容：　7 张 UML 图 • 12 个界面 • 设计报告

建模工具：PlantUML（源文件.puml 随报告提交）• 界面：HTML/CSS 设计稿，按 390×844 与 1440×900 两种画布输出

目录

1 项目概述 **2**

1.1 项目背景 2

1.2 设计输入 2

2 功能模块划分 **2**

2.1 模块划分结果 2

2.2 模块之间的交互 3

3 软件架构设计（**UML**） **4**

3.1 类图 4

3.2 组件图 5

3.3 部署图 6

3.4 用例图 7

3.5 活动图 8

3.6 状态图 9

3.7 时序图 10

3.8 模型之间的一致性 11

4 界面设计 **11**

4.1 设计规范 11

4.2 顾客端 12

4.3 商家端与骑手端 14

4.4 平台管理后台 15

5 **AI** 辅助软件设计 **16**

5.1 使用场景 16

5.2 生成结果 17

5.3 优缺点分析 17

5.4 人工修改与优化 17

5.5 关于”人只做检查校对”的思考 18

6 版本管理 **18**

7 小结 **19**

1 项目概述

1.1 项目背景

本项目是面向公众的城市外卖平台，覆盖”顾客点餐—商家接单出餐—骑手配送—平台监管”全链路。与园区内部订餐系统相比，公众平台有三个特征：配送半径以 3 公里为界、按距离分段计价，订单在空间上分散，骑手需抢单并规划路线；商家类型从连锁快餐到街边小店跨度很大，菜单与营业时间差异显著，店铺和菜品模型必须足够通用；参与方是互不相识的社会用户，平台需承担商家审核、退款仲裁与超时兜底，支付环节按 REQ-024 以模拟支付替代真实渠道。

这三条特征决定了设计的两个重点：订单状态机必须能自愈——高峰期商家来不及接单时由系统兜底，超时自动取消并全额退款；界面必须在极短的操作时间内把倒计时、距离、金额推到用户眼前。同时地址、评价、退款责任方需要独立建模，才能支撑平台侧的仲裁与风控。

1.2 设计输入

设计输入为实验一交付的需求文档，共 26 条需求，其中 25 条已确认，REQ-026（配送费计算规则）待确认，本设计按”起步价 + 距离分段计价”给出可用方案。

需求范围	内容	在设计中的落点
REQ-001 ~ 006	注册登录、角色权限、营业状态、库存、店铺与购物车	四类用户继承体系、商家域模型、顾客端浏览链路
REQ-007 ~ 012	下单、状态流转、接拒单、配送、送达、退款	订单状态机、活动图、时序图的三个参与者
REQ-013 ~ 014、022	经营统计、操作日志、管理员处置	管理后台的看板与审计页
REQ-015 ~ 021	性能、一致性、信息安全、幂等、超时兜底、地址校验	组件图分层、定时任务组件与异常分支
REQ-023 ~ 026	易用性、外部服务约束、数据保留、配送费规则	界面规范、模拟支付、部署图中的第三方节点

2 功能模块划分

2.1 模块划分结果

划分采用”按业务能力拆分后端、按角色聚合前端”的思路：后端切成 8 个模块，各自拥有独立的状态与边界；前端按角色聚合成 4 个客户端，因为四类角色的操作习惯差异极大（顾客低频浏览、商家高频处理、骑手单手滑动、管理员桌面批量操作），合成一套界面必然互相拖累。

模块	职责	对应需求	对应界面
M1 用户与权限	注册登录、角色鉴权、账号状态、收货地址	001、002、017、020	01

模块	职责	对应需求	对应界面
M2 店铺与菜单	营业状态、菜品分类与库存、店铺检索	003、004、005	02、03、08
M3 购物车与下单	购物车、价格快照、售罄校验、重复提交防护	006、007、018、023、026	03、04
M4 支付与订单状态	模拟支付、订单状态机、状态流水、超时兜底	007、008、016、019、024	04、05
M5 商家接单	接拒单、出餐、本店订单与经营统计	009、013、019	07、08
M6 配送	配送单生成、派单与抢单、取餐送达、异常上报	010、011、021	09、10
M7 售后与退款	退款申请、可退金额计算、时限处理、平台仲裁	012、016、021	06、12
M8 平台管理	商家入驻审核、违规处置、运营看板、操作日志	013、014、022	11、12

表 1　模块与需求的追溯矩阵，M1 ~ M8 覆盖 REQ-001 ~ REQ-026 全部需求

2.2 模块之间的交互

模块之间只通过订单服务这一条主干交互：顾客下单、商家接单出餐、骑手抢单送达、系统超时兜底，都是向订单服务发起一次状态迁移请求，由订单服务校验当前状态是否允许该迁移，再通知相关模块。状态只有一个权威来源，“商家说已出餐、骑手说还没取餐”这类冲突才能判定。代价是订单服务成为热点，因此组件图为它单独配置了库存预占与抢单锁。

库存校验在顾客端和订单服务各做一次，但意义不同：店铺页的校验是体验性的，把售罄菜品置灰并自动移出购物车；结算时订单服务的校验才是权威的，用缓存预占库存并对售罄返回冲突。只有前者的系统会超卖，只有后者的系统会让用户在结算页才失败。

3 软件架构设计（UML）

本章按”结构 → 功能 → 行为 → 交互”给出 7 张图，覆盖结构性、行为性、交互性三类。

3.1 类图

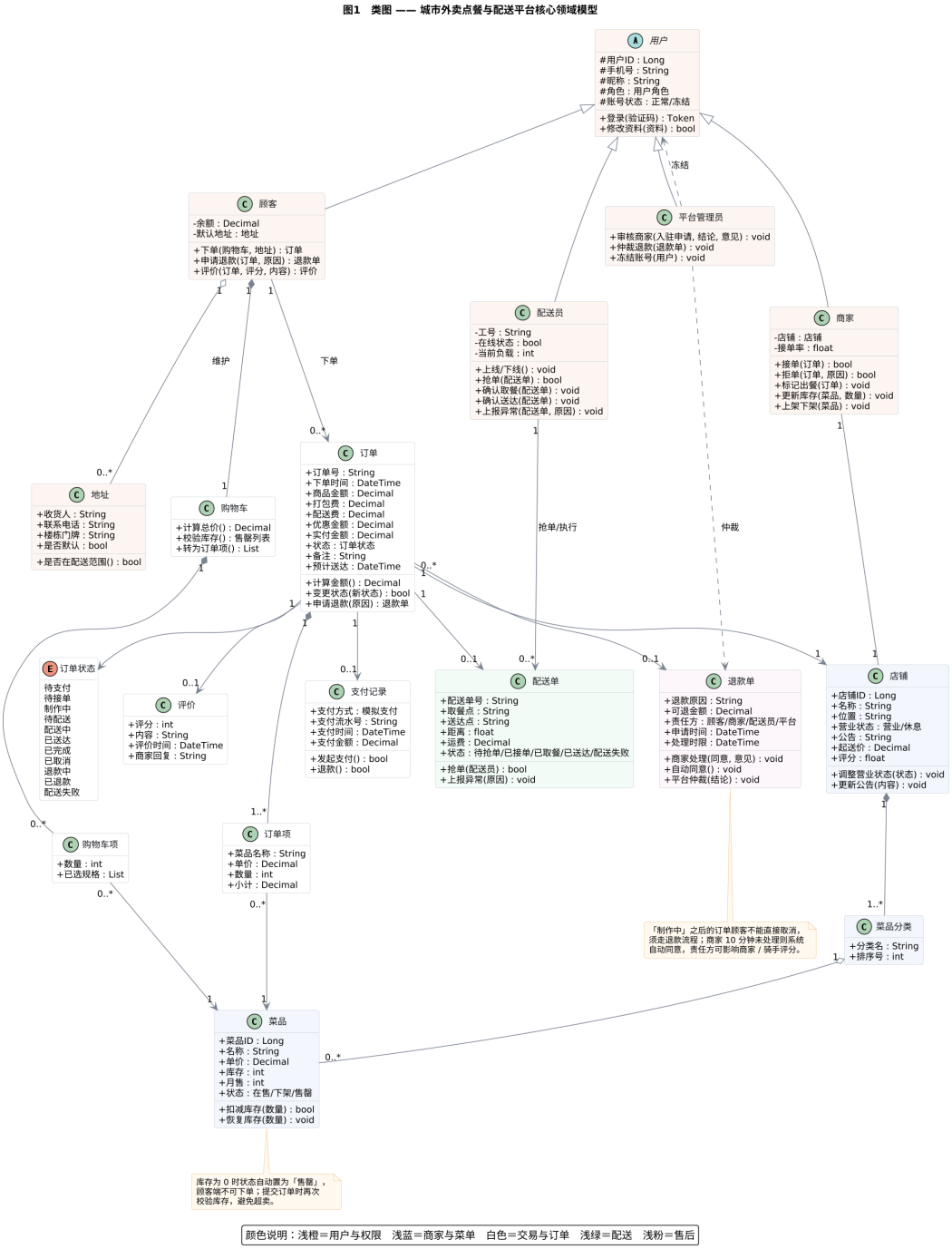


图 1　类图——核心领域模型（源文件 UML 类图.puml）

共 18 个类与 1 个枚举，按五个业务域着色：用户与权限、商家与菜单、交易与订单、配送、售后。

- 用继承表达角色，不用”角色”字段分支。四类用户的属性差异很小，但操作差异很大：只有商家能标记出餐，只有配送员能抢单。把操作声明在子类上，权限检查就变成编译期可判定的类型问题。
- 订单与订单项是组合关系。订单项单价是下单时刻的价格快照（REQ-007），而不是对菜品的实时查询，否则商家改价会让历史订单金额跟着变。
- 订单状态是枚举而不是布尔标志位。状态图的 11 个状态即枚举的 11 个字面量；若用三个布尔标志会有 8 种组合、其中至少 4 种非法（如”已送达但未支付”）。
- 配送单与订单分离。一笔订单可能经历多个配送单（异常后重新派单），一对多是常态，配送单因此有独立的生命周期。
- 退款单独立成类。它有原因、可退金额、责任方、申请时间、处理时限五个自有属性和三条处理路径，本质是一个小状态机。

3.2 组件图

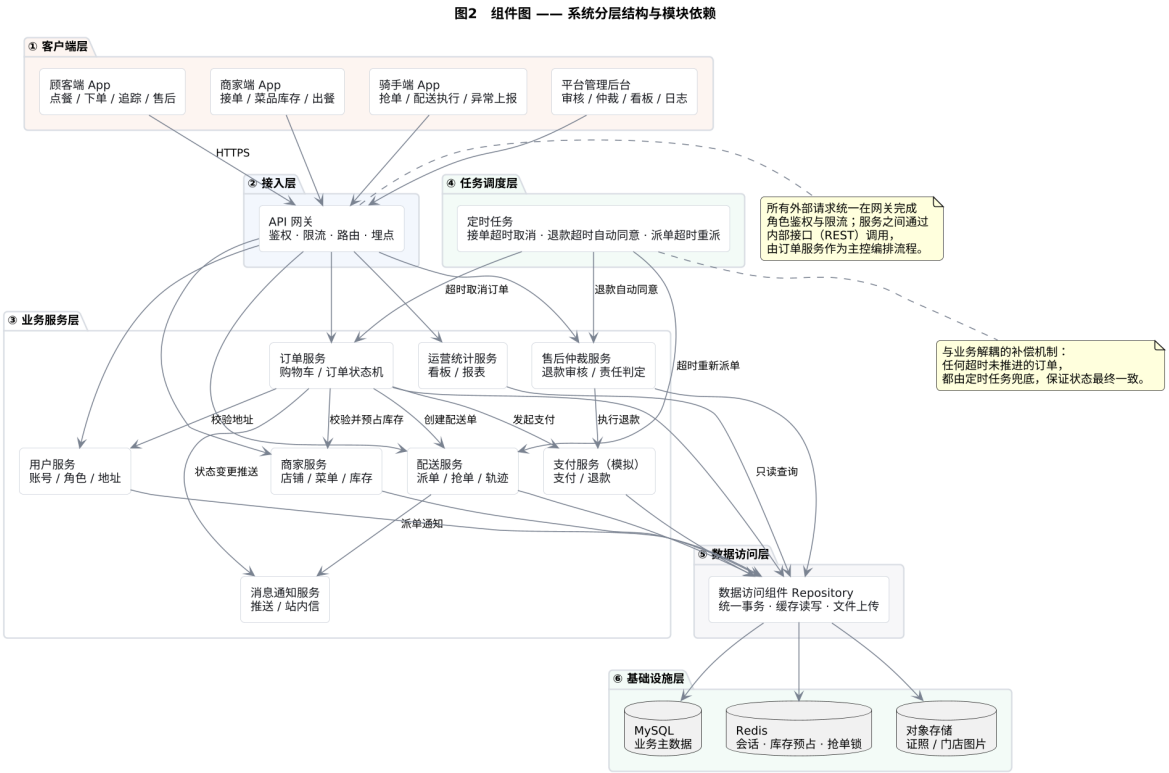


图 2　组件图——分层结构与模块依赖（源文件 UML 组件图.puml）

- 六层结构单向依赖：客户端 → 网关 → 业务服务 → 任务调度 → 数据访问 → 基础设施。依赖只能向下，下层组件可以独立替换，例如把模拟支付换成真实支付只影响支付服务一个组件。
- 任务调度层与业务服务层解耦。订单超时取消、退款超时自动同意、派单超时重派三件事的共同点是”没有请求触发”，必须由外部时钟驱动；抽成独立一层后，业务服务只需保证被调用时行为正确。

- 加入 **Repository** 数据访问组件。业务服务不直接连数据库，把事务边界、缓存读写、抢单锁收敛到一处，保证抢单操作的原子性。

3.3 部署图

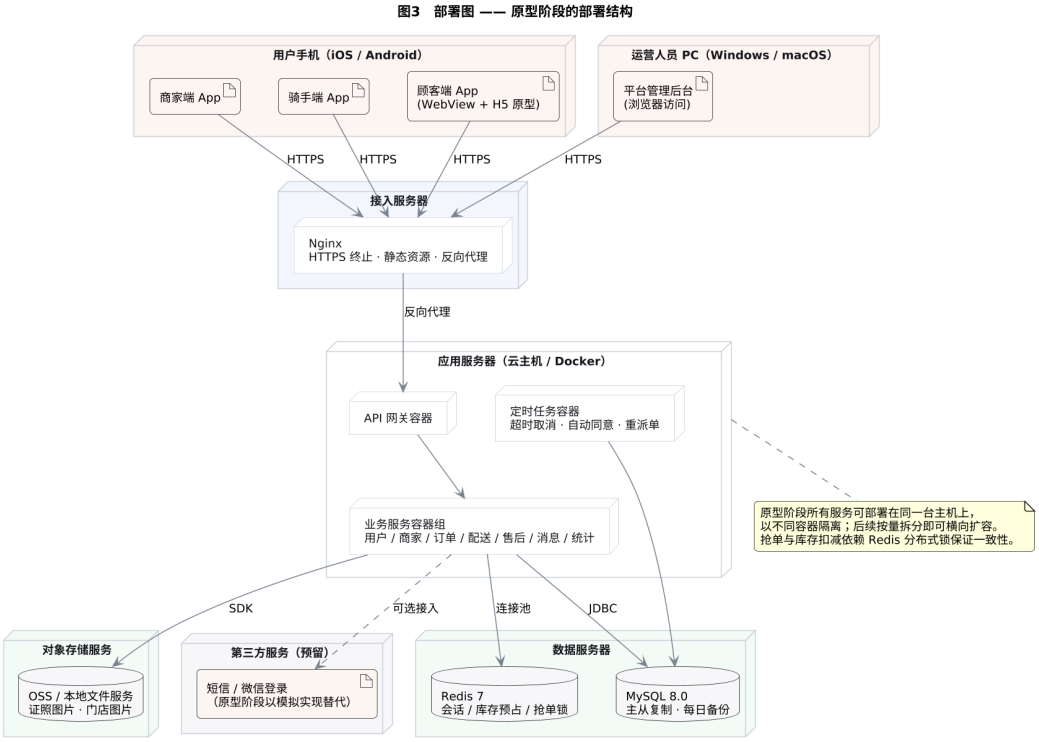


图 3　部署图——原型阶段的物理部署（源文件 UML 部署图.puml）

部署图反映课程原型阶段的规模：一台 Nginx 做反向代理与静态资源、一台应用服务器运行 API 网关与业务服务容器组、一台数据服务器运行 MySQL 与 Redis，对象存储单独放置证照与门店图片。图中没有画微服务集群——部署图描述的是”软件要跑在什么上”，画出项目不会部署的集群只会让图失去可信度。第三方支付节点以虚线标注为预留，对应 REQ-024。

3.4 用例图

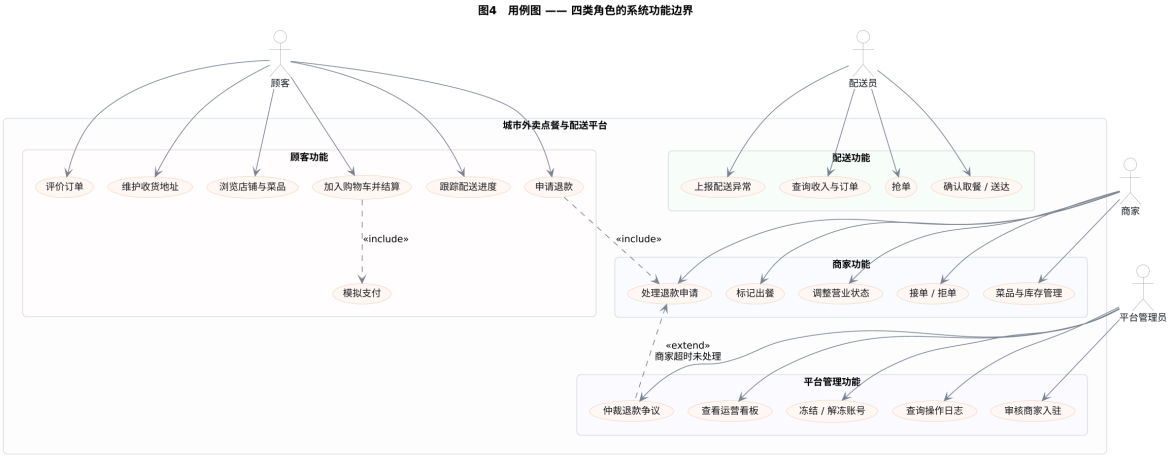


图 4　用例图——四类角色的功能边界（源文件 UML 用例图.puml）

- 系统边界内按角色分成四个功能区块，“谁能做什么”一眼可见，直接对应 REQ-002 的权限矩阵。
- 只保留两个 include 与一个 extend：结算包含支付、申请退款包含商家处理，“商家超时未处理”扩展为平台仲裁退款争议。用例全部用 include 织成网，图就退化成一团线。
- 没有把”系统”画成参与者：定时任务是系统自身行为，不是外部参与者，它出现在活动图与组件图中。

3.5 活动图

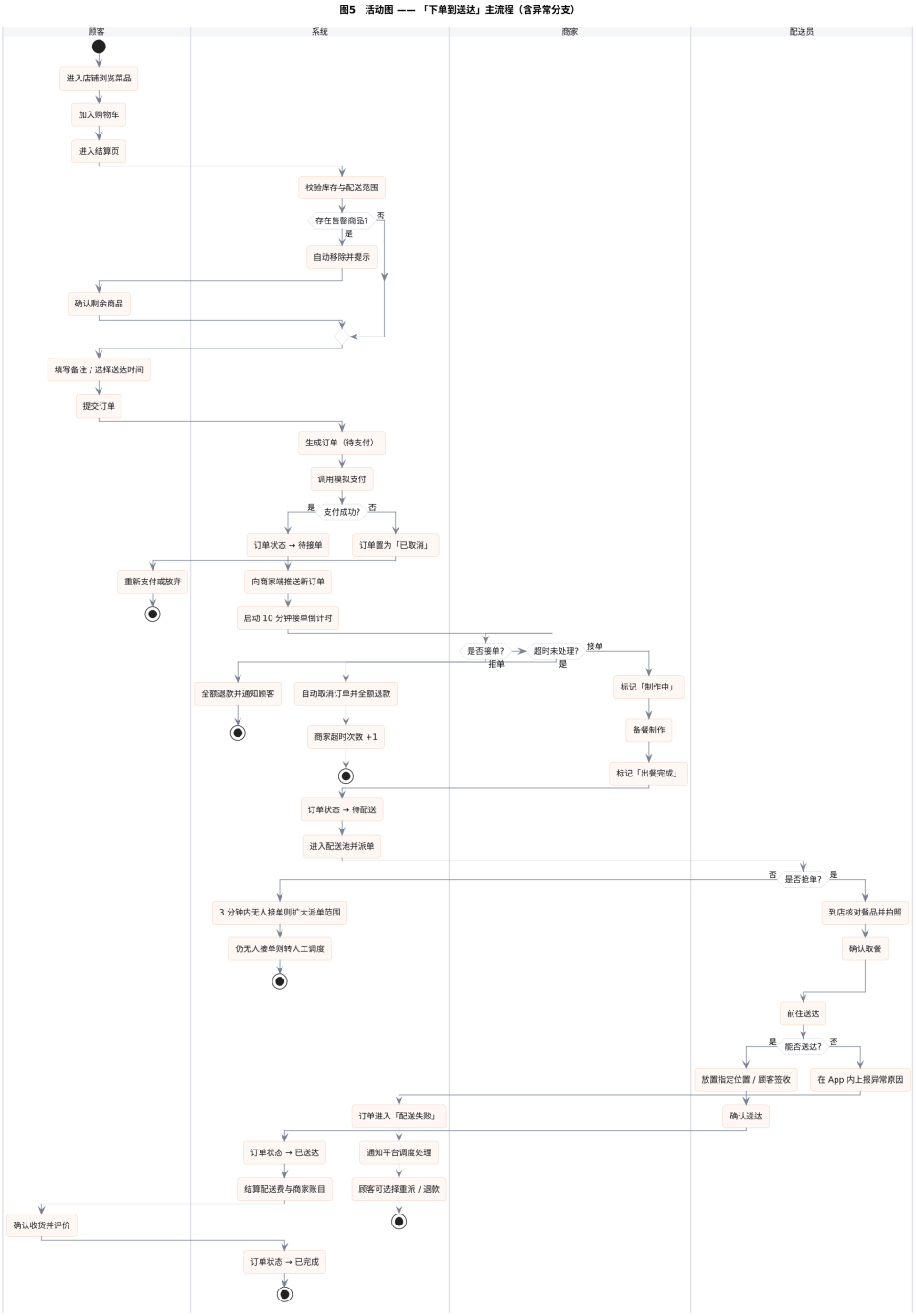


图 5　活动图——” 下单到送达” 主流程与异常分支（源文件 UML 活动图.puml）

- 用泳道而非普通流程图：泳道同时回答”做什么”和”谁来做”，任何一个跨泳道的箭头都是一次接口调用，正好对照组件图的依赖。
- 异常分支是这张图的主角。主流程只有 6 步，难点全在 6 个失败出口：售罄、支付失败、商家

拒单、接单超时、无人抢单、配送失败。每条失败路径都画到终态并注明补偿动作（全额退款、次数加一、扩大派单范围、转人工调度）。

3.6 状态图

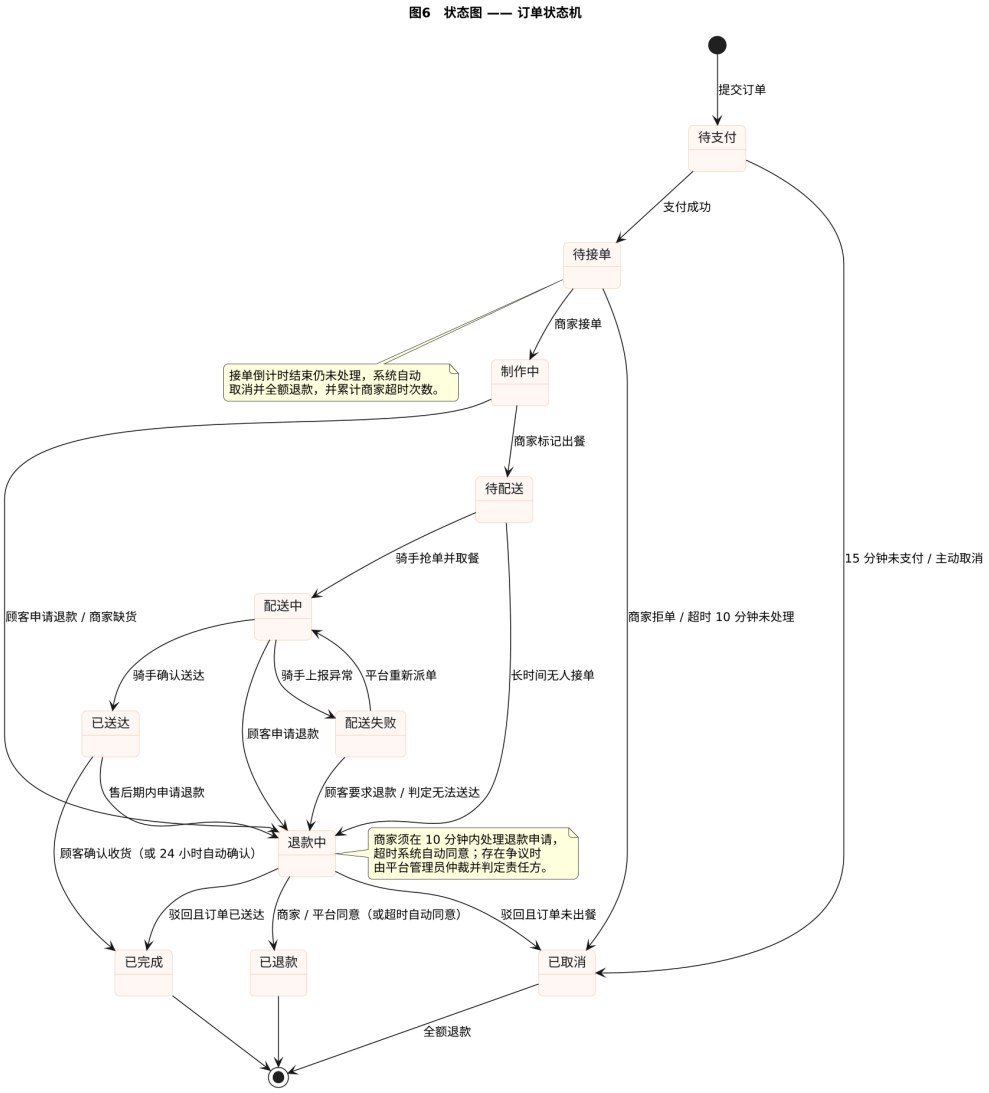


图 6　状态图——订单状态机（源文件 UML 状态图.puml）

- 信息密度最高、最应作为实现依据的一张：转移标签写的是触发条件与守卫条件，可直接翻译成状态机的迁移表。
- 状态机是唯一权威。商家端、骑手端、后端都不能直接改订单状态，只能发起迁移请求，由状态机校验合法性，“对未出餐订单点确认送达”会在服务端被拒绝。
- 画出了容易漏掉的回流边：配送失败 → 配送中（重新派单）、退款中 → 已完成 / 已取消（退款被驳回）。只画单向推进，开发时”退款被驳回后订单该去哪”就无解。

3.7 时序图

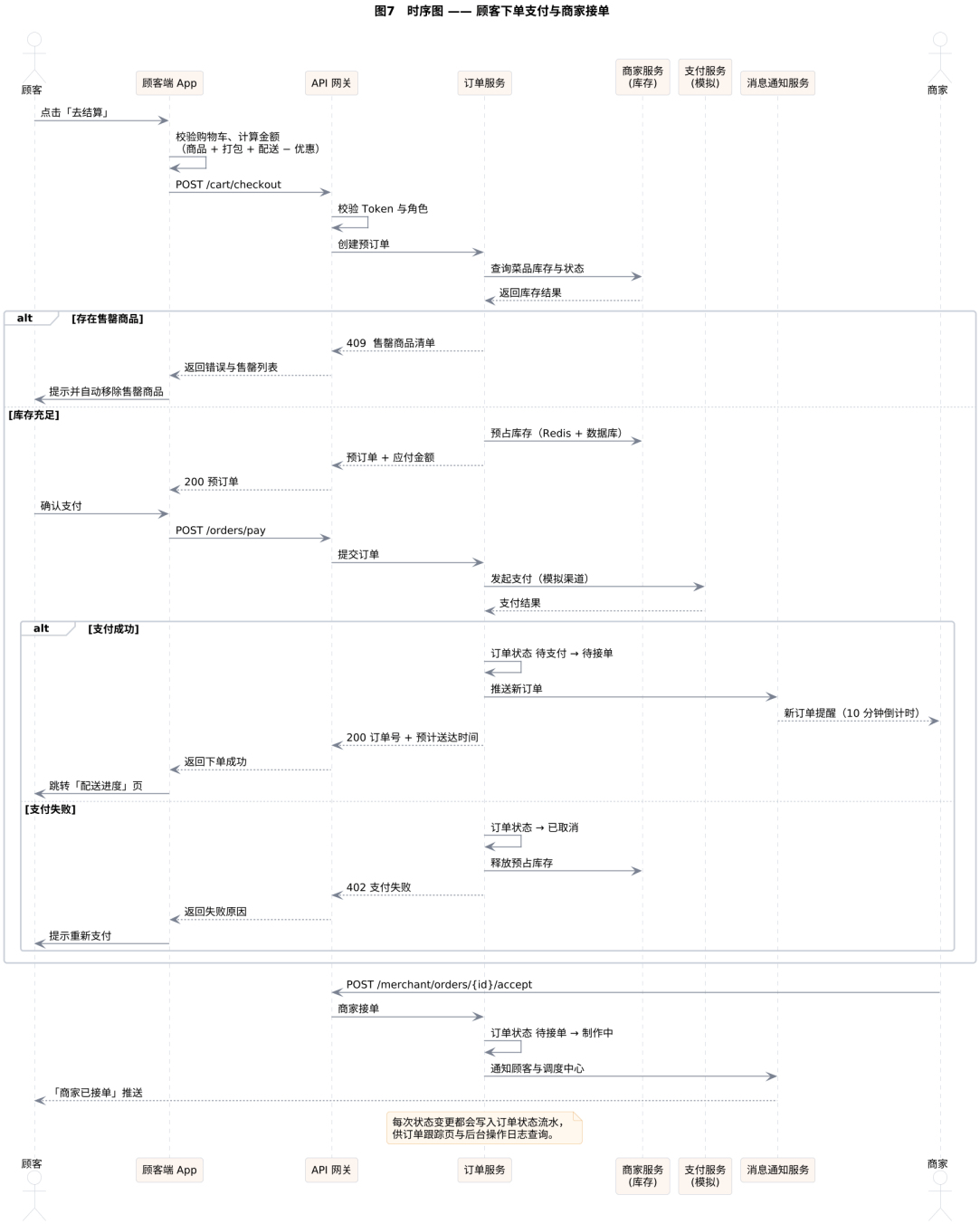


图 7　时序图——顾客下单支付与商家接单（源文件 UML 时序图.puml）

- 选的是最容易出错的一次交互：下单与支付。这条链路串起 6 个参与者，库存校验、库存预占、支付、状态迁移四个动作有严格先后顺序，用文字描述极易产生歧义。
- 用两层嵌套 alt 把”售罄”和”支付失败”画在正确位置：售罄发生下单之前，订单不生成；支付失败发生在下单之后，订单已生成需取消并释放库存。两者补偿方式不同，画在同一分支里几乎必然写错。

- 商家接单是独立的一小节交互，说明顾客侧与商家侧不是一次长事务，而是由消息推送解耦的两次交互，这决定了高峰期系统的可用性。

3.8 模型之间的一致性

七张图是同一系统的七个切面，本轮对四组跨图元素做了显式对齐：

对齐项	涉及图	对齐结论
订单状态字面量	类图枚举、状态图、界面文案	统一为 11 个状态，待支付 / 待接单 / 制作中 / 待配送 / 配送中 / 已送达 / 已完成 / 已取消 / 退款中 / 已退款 / 配送失败
参与者与组件	时序图、组件图	时序图每个参与者都能在组件图中找到实体，业务服务都能落到某条时序上
超时时限	状态图、活动图、界面、需求	待支付 15 分钟、接单 10 分钟、退款处理 10 分钟、派单 3 分钟，四处一致
角色与用例	用例图、类图、界面	顾客 7 个用例、商家 5 个、骑手 4 个、管理员 5 个，与 12 个界面对应

4 界面设计

4.1 设计规范

12 个界面按 4 个客户端分组，共用一套设计令牌，由 `ui.css` 统一管理。目的不是”看起来统一”，而是让同类信息在不同端上的视觉编码一致：顾客在手机上看到的橙色按钮与商家在平板上看到的确认按钮，含义都是”主操作”。

令牌	取值	使用规则
品牌色	#FF6B2C / #E8501A	只用于主操作、当前状态、价格与倒计时，一屏内主操作不超过一个
语义色	绿 #12B76A ・蓝 #2E6BE6 ・橙 #F79009 ・红 #E5484D	绿为完成、蓝为进行中、橙为待处理、红为超时与失败
文字与数字	#15171C / #5A6270 / #8A909C	标题、正文、辅助信息；金额用等宽数字避免跳动
圆角与字号	卡片 12px、按钮 10px；11 ~ 21px 阶梯	圆角只用于可点击容器；手机端最小字号 11px

另有一条贯穿全部界面的原则：每个界面只回答用户当下的一个问题。店铺页回答”这家店有什么、贵不贵”，结算页回答”这一单付多少、什么时候到”，配送任务页回答”现在该去哪”。不能

回答这个问题的信息即使有用也放到下一层。

4.2 顾客端

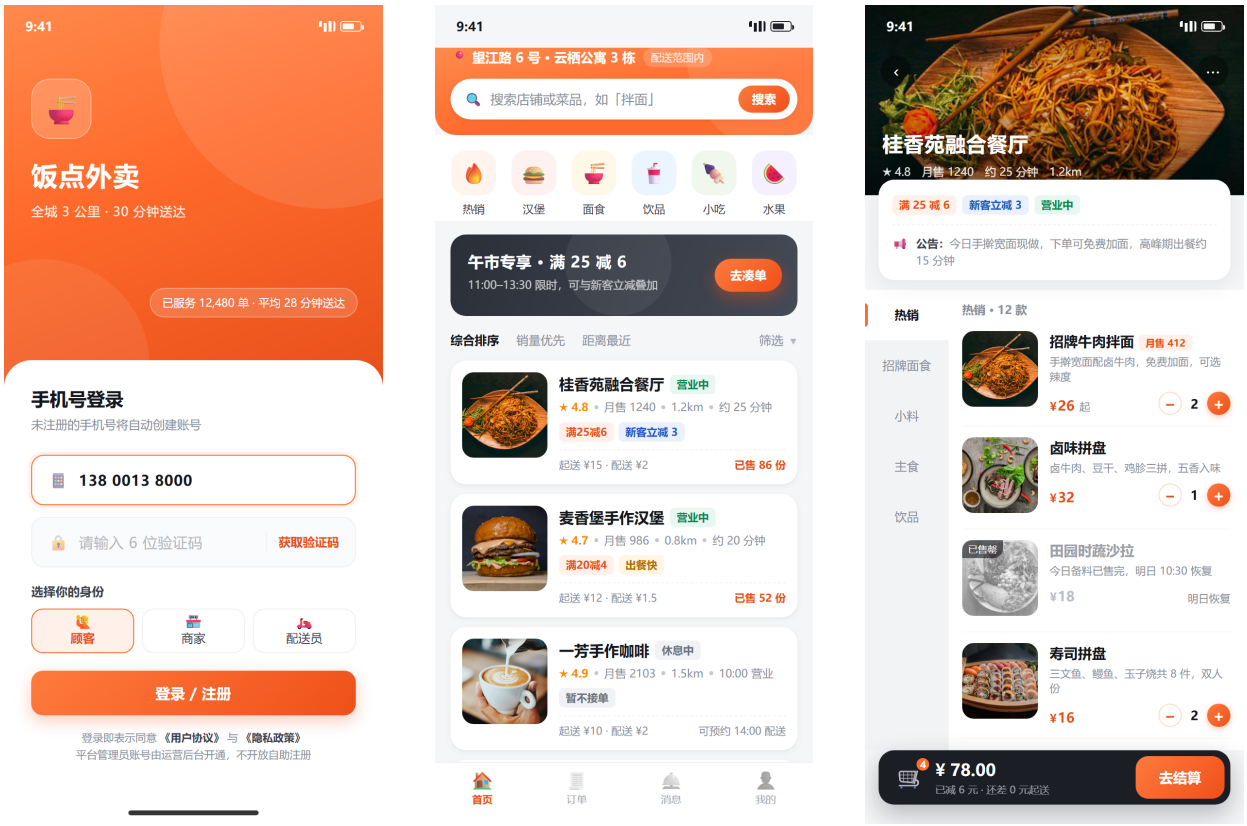


图 8　顾客端（一）：登录注册、附近店铺、店铺与购物车

- **01** 登录注册（REQ-001、002）：手机号加验证码单步登录，核心是身份选择器——三类角色同一入口登录，登录后分别进入顾客首页、商家工作台、骑手抢单大厅。底部写明平台管理员账号由运营后台开通、不开放自助注册，把权限边界放在用户看得见的地方。
- **02** 首页 · 附近店铺（REQ-005）：按距离、销量、评分排序的店铺卡片流，每张卡片给出店名与评分、招牌图、起送价与配送费、预计送达时间四个决策要素。真正决定用户点不点的是”要多久送到”，所以配送时间放在卡片右下角而不是详情页。休息中的店铺整体降饱和并标注”休息中”。
- **03** 店铺与购物车（REQ-003 ~ 006）：左侧分类锚点加右侧菜品流，适配单手操作。售罄菜品置灰且不可加购；购物车条实时显示”还差 X 元起送”，把门槛变成进度提示；店铺公告与营业状态顶部常驻。购物车条只在有商品时升起，空车不占屏。

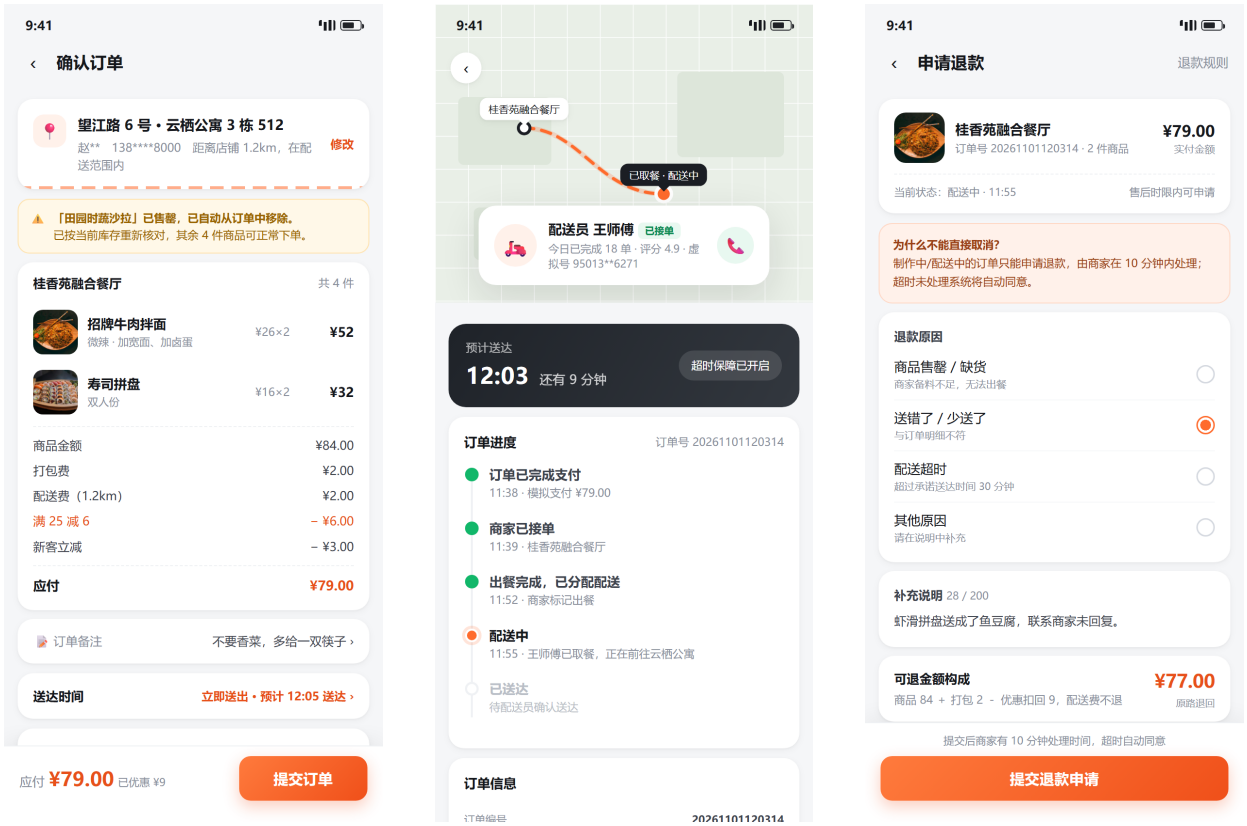


图 9　顾客端（二）：确认订单、配送进度、申请退款

- **04** 确认订单（REQ-007、018、020、023、026）：信息最密的一屏，按重要性排成四段——送到哪、买了什么、花多少钱、怎么付。地址行直接显示”距离店铺 1.2km，在配送范围内”，把地址校验前置到确认之前；进入结算时重新校验库存，售罄商品自动移除并给出黄色提示条；费用逐项列出，配送费标注距离，为待确认的计费规则留出解释位置；支付方式明确标注”模拟支付”并说明不接入真实渠道。
- **05** 配送进度（REQ-008、011）：上半屏地图回答”还有多远”，下半屏时间线回答”到哪一步了、为什么慢”。时间线每一行都能在状态机中找到对应迁移，界面是状态机的可视化而非另写文案；骑手信息显示虚拟号，对应 REQ-017 的信息安全要求；底部保留配送中发起退款的入口，正是状态图中”配送中 → 退款中”那条边的来源。
- **06** 申请退款（REQ-012、016）：设计重点是把规则先说清楚。顶部提示条写明”制作中、配送中的订单需商家 10 分钟内响应，超时系统自动同意”，与状态图的实现是同一套；原因用单选列表而非下拉框，四个高频原因一屏可见；底部金额条拆开算给用户看：商品 84 加打包 2 减优惠扣回 9 等于 77，配送费不退。

4.3 商家端与骑手端

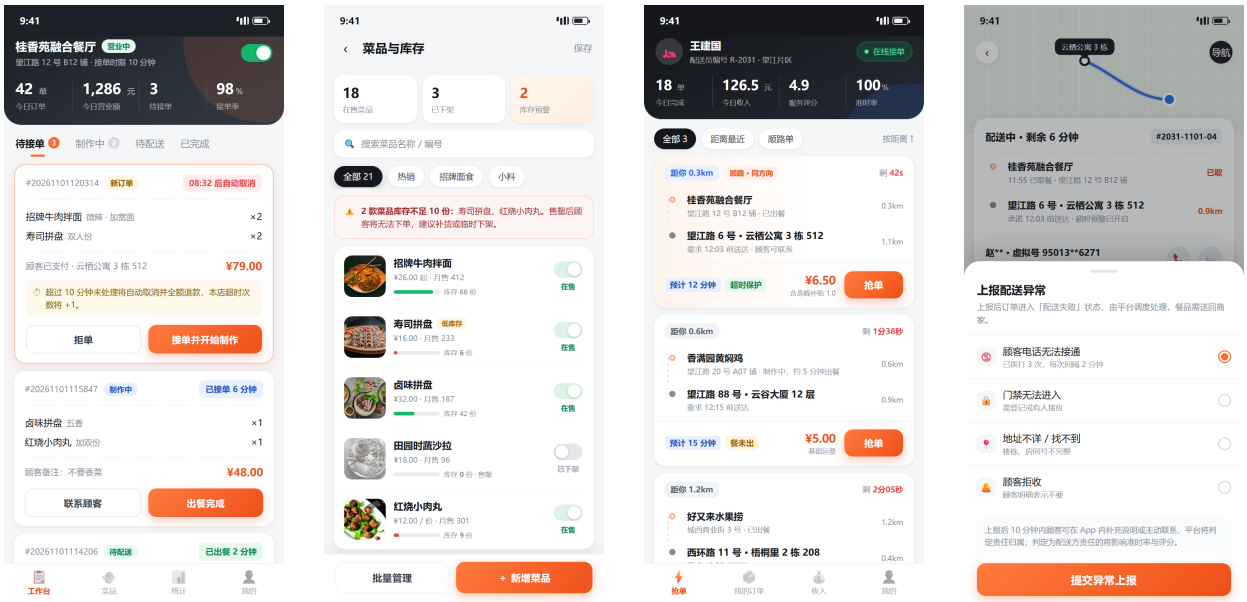


图 10　商家端与骑手端：商家工作台、菜品管理、抢单大厅、配送任务

- **07** 商家工作台（REQ-009、013、019）：关键指标是处理一单的时间。整屏按待接单、制作中、待配送分段，默认停在待接单。倒计时印在卡片右上角并用红色强调，卡片本身带红色描边，高峰期扫一眼颜色就知道哪单最急；提示条写明”超过 10 分钟未处理将自动取消并全额退款，本店超时次数加一”，只写倒计时是威胁，写清后果才是规则；拒单与接单按钮同尺寸反色，避免误触。
- **08** 菜品与库存管理（REQ-004）：核心不是增删改，而是让商家开店时十秒内判断哪些菜要补货。每项用横向水位条表示库存比例，低于 15% 变橙并打”低库存”标签，零库存置灰标注售罄；右侧在售开关支持一键下架，高频动作一步完成；顶部汇总在售、预警、下架三个数字。
- **09** 抢单大厅（REQ-010）：骑手在骑行途中看手机通常不超过 3 秒，因此只保留四个数字——取餐距离、送达距离、总价、预计用时，加一条橙绿双点路线示意。收益构成写清基础运费、高峰补贴、远距离补贴，因为骑手对”这单值不值”的敏感度远高于对订单内容的兴趣。
- **10** 配送任务与异常上报（REQ-011、021）：唯一需要边骑车边操作的页面，主按钮固定在底部、按单手拇指可达设计。四步进度条标出当前步骤与下一步动作；上报异常是弱按钮，不与主流程抢视觉权重；弹层给出四个枚举化的异常原因，每个原因下方说明处理后果，枚举化便于后续责任判定与统计。

4.4 平台管理后台

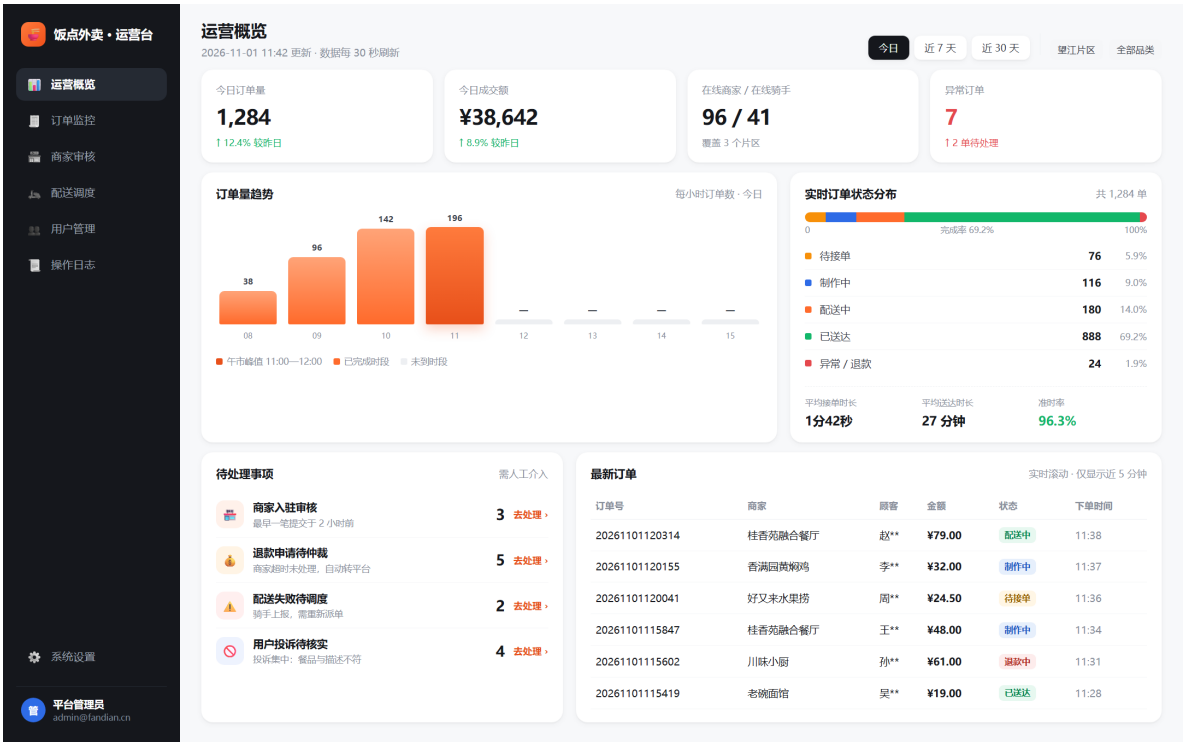


图 11　运营看板：KPI、订单趋势、状态分布与待处理事项（REQ-013）

管理后台是低频、高信息密度、批量处理的场景，因此采用 1440×900 桌面画布与左侧固定导航，而不是把手机界面放大。看板回答管理者每天打开系统后的第一个问题——今天有没有异常，所以把”待处理事项”放在第一屏可见位置，而不是把最多空间留给趋势图；四个 KPI 都带同比标识，因为单看绝对值没有决策意义。



图 12　商家审核与操作日志：审核队列、证照材料与结论留痕（REQ-014、022）

审核页分三栏：左侧待审核队列、中间详情（营业执照、食品经营许可证、门头照三张材料与逐项检查清单）、右侧操作区。审核是最需要留痕的操作：驳回必须填写原因，原因作为操作日志永久保留；页面底部直接给出操作日志表，不跳转也能确认刚才的操作是否生效；日志中保留失败记录，因为审计的价值恰在于记录被拒绝的操作。

四个端的视觉差异是有意为之：顾客端用大图与留白营造食欲，商家端用高对比色块传达紧迫，骑手端用极简大字保证骑行中的可读性，管理后台用密集表格容纳信息量。它们共享同一套颜色语义与圆角规则，但不共享布局——统一的是语言，不是版式。

5　AI 辅助软件设计

本次设计中，大语言模型参与了领域建模、架构图绘制、界面生成、渲染脚本编写与文字整理五个环节。本章按使用场景、生成结果、优缺点、修改优化、反思的顺序记录，包括失败的尝试。

5.1　使用场景

场景	输入	产出	人工承担的部分
S1 领域建模	需求文档全文，要求按业务域给出类图	类图源码与类清单	判断哪些类该存在、关系基数是否正确
S2 架构图	模块划分表，要求画出六类图	6 张图源码	决定画到多细、剔除不成立的依赖

场景	输入	产出	人工承担的部分
S3 界面设计	业务线说明与设计令牌	12 个界面 HTML/CSS	信息层级判断、业务规则自洽、视觉取舍
S4 工程脚本	渲染 PlantUML 并校验结果	PowerShell 脚本	定位失败原因、确定编码方案
S5 文档整理	各图与各屏要点	报告初稿	核对每条论断是否有图或需求支撑

5.2 生成结果

五个场景的产出都”能跑”，质量差异很大，这个差异本身是本章最有价值的发现。

- S4 脚本与 S5 文档几乎一次可用，人工只需少量修正。这两类任务的正确性可以被立即验证（脚本跑没跑通、论点能否指回一张图），模型面对的是可判定的目标。
- S1、S2 的 UML 图能用但布局不可控。首轮类图 18 个类相互交叉，三条长边横穿整张图；活动图泳道在条件分支后错位，导致商家动作出现在顾客列。模型能写出语法正确的 PlantUML，但无法预判布局引擎的排布结果。
- S3 界面视觉完成度高、业务自洽度低。初稿观感不错，复核却发现结算页写”其余 3 件商品”而同页写着”共 4 件”；商品行只给”×2 ¥52”会被误读为单价；商家工作台写”已指定王师傅”，与需求中”骑手抢单”的派单模型矛盾。

5.3 优缺点分析

优点。覆盖面完整，26 条需求一次性映射到 8 个模块、18 个类、21 个用例，人工做同样的梳理至少需要数小时且容易漏项；术语一致，同一批生成内容里”待接单、制作中、待配送”的写法几乎不会漂移；可快速给出多个方案，类图的三种排布、活动图的两种泳道组织可在几分钟内比较；把形式从内容里剥离，设计者只需判断关系对不对。

缺点。不知道物理约束，模型不会主动考虑”这一屏只有 844 像素高”，首轮多个界面内容溢出，必须靠渲染截图才能发现；业务规则不自洽，单看每句文案都对，放在一起就矛盾，因为生成这一页时并不知道上一页写了什么；视觉平均主义，所有卡片权重相近，缺少”这一屏最重要的事是什么”的决断；布局不可控，UML 自动排布需反复试；倾向补全而非删减，克制需要人来加。

5.4 人工修改与优化

下列修改均已提交到 git，提交信息带 [人工修改] 前缀。

对象	AI 初稿的问题	人工修改后的结果
类图	用包框分组，类框越出边界；角色子类之间有三条长边横穿全图	去掉包框，改为按业务域着色加图例；删除跨模块长边，改为经中间类间接关联

对象	AI 初稿的问题	人工修改后的结果
类图	枚举缺”配送失败”；评价、地址两个类型在图中不存在	补入枚举值，补建两个类并连上关联
组件图	八个服务平铺，依赖线交织成网，看不出层次	重写为六个带序号的分层，加入 Repository 抽象收敛依赖
活动图	泳道在条件分支后丢失，商家动作出现在顾客列	在分支开头与结束处重新声明泳道，逐分支核对归属
状态图	只画正向推进，没有退款被驳回后的回流边	补入两条驳回边与配送失败后的重派边
04 结算页	售罄提示与总件数矛盾；“×2 ¥52”会被读成单价	统一为 4 件，显式写成 ¥26×2 → ¥52
08 菜品页	库存水位条不可见，子元素的宽高被忽略	为轨道与填充增加块级显示，水位条恢复

5.5 关于”人只做检查校对”的思考

把本轮的人工工作按性质分为三层，只有第一层可被工作流化。

- 可判定层：金额加起来对不对、状态枚举是否一致、PNG 是不是真图片、页面有没有溢出。有唯一答案，写成断言即可。本轮已落地两条：渲染结果的图片魔数校验、逐屏比对金额等式。
- 语义裁决层：售罄后”还剩几件”指件数还是种类、退款时优惠要不要扣回、配送员能否看到门牌号。需求文档没写死，模型只能猜，猜得再合理也是替甲方做决定，必须显式交还给人并记录为需求变更。
- 价值判断层：结算页”预计送达时间”该多大字号、这一屏最重要的信息是什么、图该画到多细。没有对错只有取舍，AI 能给多个都可行的方案，选择取决于产品想成为什么。

本轮形成了一个可复用的闭环，人工介入点被压缩到两处：一是约束前置，把需求编号、设计令牌、画布尺寸一次性写进上下文，让模型带着约束生成，返工率因此大幅下降；二是生成、渲染、回读、校验，模型写界面，脚本用无头浏览器渲染成图，再把图读回给模型——纯文本状态下模型看不见自己的输出，一旦能看到渲染结果，溢出与对比度问题就能自行发现。可自动化的上限由需求的可判定程度决定：金额检查能自动化，是因为实验一把计价规则写成了可判定的等式；REQ-026 含糊，检查就无从下手。要让工作流跑起来，先得有人做最不可自动化的那部分——它不会消灭人工，只是把人推到更上游。

6 版本管理

全部设计产物（PlantUML 源码、界面 HTML/CSS、渲染脚本、本报告）纳入 git 管理，并用提交信息区分 AI 生成与人工修改：`[AI 生成]` 表示模型一次性产出、尚未经人工复核的内容；`[人工修改]` 表示复核后的修正，信息中写明问题、原因与结果；无前缀为脚手架与报告本身等非设计产物。

提交	类型	内容
基线	—	实验要求与实验一需求文档
UML 初稿	[AI 生成]	7 张图源码
界面初稿	[AI 生成]	设计令牌与 12 个界面
界面审查	[人工修改]	修正业务矛盾与金额歧义
类图审查	[人工修改]	补全缺失的领域对象与状态值
场景调整	[人工修改]	由园区场景改为面向公众的城市外卖平台

目录结构如下，提交包只含报告、UML 组与界面图。

```
lab2/
├── requirements.md          实验二要求
├── design/
│   ├── uml/                 7 组 .puml 源文件与导出 PNG
│   ├── ui/                  12 个界面 HTML 与 ui.css 设计令牌
│   ├── out/                 渲染后的界面 PNG
│   ├── render-*.ps1         PlantUML 渲染与界面截图脚本
│   ├── report.md            本报告源文件
│   └── report.pdf           由 Markdown 渲染的最终报告
└── 实验2_241220075_赵力丹/
    ├── 实验报告.pdf
    ├── UML/
    └── UI界面/
```

7 小结

本次实验把实验一的 26 条需求落成 7 张互相一致的 UML 图与 12 个可用界面：类图给出领域模型与状态枚举，组件图给出分层与依赖，用例图划定四类角色的边界，活动图与状态图覆盖主流程与异常分支，时序图钉死支付链路。

四个端共用一套颜色语义与圆角规则，版式各自为政：顾客端重氛围、商家端重紧迫、骑手端重大字与单手操作、管理后台重信息密度，每一屏只回答一个问题。

AI 让覆盖面从”做三分之一”变成”做全”，也暴露了不知道物理约束、规则不自洽、缺少主次判断三个短板。起作用的不是让 AI 写，而是把它放进生成、渲染、回读、校验、裁决的闭环——人的位置由此从执行者变成约束的定义者与歧义的裁决者。